BACKEND ENGINEER · PORTFOLIO

안녕하세요,
신입 백엔드 개발자
엄주호입니다.

꾸준히 공부하고 발전하는 과정을 좋아합니다. 새로운 기술을 적용하거나, 발생할 수 있는 다양한 유저 시나리오에서 트러블슈팅과 프로세스 개선을 해내는 순간에 재미를 느낍니다.

5+	N1	1/9
Projects	JLPT 153점	LLM 비용 절감 (NewVent)

EXPERIENCE

LG U+ 유레카 SW 교육과정 4기 - 백엔드

2026.04.06 - 2026.10.28

Servlet 라이프사이클 같은 원천 기술부터 Spring Boot, JPA까지 더 본질적으로 학습하고 싶어 입과했습니다.
NewVent, HighFive 등 여러 프로젝트를 진행했습니다.

株式会社テクノプロ・IT - 일본 해외 근무

2024.02 - 2024.10

- LMS(학습관리시스템) SaaS 관리자 페이지로 사내 교육과정 등록·운영
- 자회사 요청 기반 신입사원 연수 등 온라인 강의 등록, 임직원 수강 권한 부여
- Excel로 강의 등록·수강 현황 데이터 취합, 월별 통계 그래프 작성 및 정기 보고

SKILLS

Language	Java · Go · Python · JavaScript · C/C++
Framework	Spring Boot · Spring Security · JPA · Gin · SQLC
Data	PostgreSQL · pgvector · MySQL · Redis
AI · Infra	AWS Bedrock · Ollama · AWS EC2 · Docker · GitHub Actions

엄주호 · uh787355@gmail.com · 010-9984-3865 github.com/UHO99 · uho99.github.io

FEATURED PROJECT · 01

NewVent

요청문 한 줄로 이벤트 페이지를 만들고, 게시·참여·추첨까지 관리하는 LLM 기반 서비스

LG U+ 유레카 · 8인 팀 · 백엔드 / LLM 모델 평가 담당 · github.com/New-Vent

문제

이벤트 페이지 한 장에도 기획 → 디자인 → 퍼블리싱을 매번 거칩니다. 문구 하나만 바꿔도 다시 디자이너와 개발자에게 돌아갑니다.

해결

"가을 응원 이벤트, 혜택은 쿠폰 3종"이라고 말하면 페이지가 생성되고, 말이나 클릭으로 고치고, 바로 게시합니다.

핵심 원칙: LLM에 맡기되, 결과는 서버가 책임진다

필터링	RAG	프롬프트	LLM	정화	검증	재시도
개인정보·모호함	유사 이벤트	레지스트리	gemma-3-27b	허용 태그만	값 보존·지어낸 값	최대 4회

담당 업무

영역	내용
LLM 출력 계약	블록·변형·팔레트를 레지스트리로 정의해 LLM의 디자인 선택 범위를 서버가 통제. 블록 4종 · 변형 191종 · 팔레트 143종 · 배치 13종 추가. 확장 항목은 요청문이 부를 때만 싣도록 PromptBuilder 정리
동작 배치기	카운트다운 등 동작을 LLM이 JSON으로 고르고 서버가 HTML에 심는 구조 설계. 정화 단계에서 동작 속성 보존 검사 추가
검증	지어낸 연도 검사, "500분께" 같은 높임말을 수량으로 판정하도록 값 검증 수정, 실패 코드를 enum 레지스트리로 통합
AI 수정	미리보기에서 고른 블록만 수정하도록 범위 제한, 선택 영역의 전체 팔레트 변경 차단
인증·인가	JWT + Refresh Token Rotation. 재발급 시 토큰 소진 → 계정 확인 → 이상 시 전체 폐기. 사용자/관리자 쿠키 분리, 공개 API 무작위 접근 수정, prod 위험 설정 기동 차단
프론트	관리자 생성·AI 수정·직접 수정·버전·게시 서버 연동, event.css 계층 분리, runtime.js

Tech

Java 21 · Spring Boot 4.1 · Spring Security · JPA · Flyway · jsoup · PostgreSQL 17 + pgvector · AWS Bedrock · EC2 · ECR · SSM · GitHub Actions OIDC · Vite · Vanilla JS

FEATURED PROJECT · 01 - DEEP DIVE

LLM 모델 선정 벤치마크

"어떤 모델을 쓸까"를 감이 아니라 측정으로 정했습니다. 저장소 최다 기여자로 실험 설계와 수행을 주도했습니다.

github.com/New-Vent/model-benchmark · Python · Ollama → AWS Bedrock · v1 ~ v11

11	10,000+	1/9
실험 버전	측정 행 (v1~v7)	동일 성능 대비 비용

최종 비교 (Bedrock, v8~v9)

모델	생성·수정	라우터	실서비스 추정 비용
gemma-3-27b ✓ 선정	만점	**24/24**	**1,119**원
claude-haiku-4.5	만점	동점	**7,883**원
gpt-oss-120b	만점	**22/24**	추론 토큰 8배 과금
qwen3-coder-30b	만점	**16/24**	-

내가 한 일

- **공통 엔진 모듈화** - 버전마다 흩어진 실행 스크립트를 base 엔진으로 통합. 프롬프트·채점기·케이스 중 하나라도 바뀌면 새 버전으로 분리해 결과가 섞이지 않게 관리
- **평가 기준 자체를 검증** - v1·v2 사이 평가 기준의 불공정을 발견해 기록하고 재측정
- **v6 단독 수행** - 문구 생성을 빼고 구조만 내게 하거나 삭제 연산 표현을 바꾸자 통과율 0%였던 축이 100%로. "모델을 바꾸는 것보다 요청 형태를 바꾸는 쪽이 효과가 컸다"
- **Bedrock 이전** - 로컬 → AWS Bedrock 이전 근거 정리, Bedrock 평가 파이프라인 구축
- **재현 가능한 기록** - 실패 유형 분석 엑셀, 문서 구조와 읽는 순서 정리, 로컬 LLM 톤앤매너 확인용 GUI

배운 것

채점기도 틀린다
채점 오류로 모델을 과소평가한 사례를 찾아 고쳤습니다.

정화 뒤를 재야 한다
모델 출력이 맞아도 정화가 망가뜨리면 제품은 깨집니다.

체급이 답은 아니다
로컬 7B가 30B보다 라우터를 잘했습니다.

FEATURED PROJECT · 02

HighFive

재고 1만 개 쿠폰에 10만 명이 동시에 몰려도 초과 발급 없이 빠르게 처리하는 선착순 쿠폰 발급 시스템

LG U+ 유레카 종합 프로젝트 · 5인 팀 · 백엔드 (저장소 최다 커밋) · github.com/UHO99/HighFive

문제

DB 비관적 락은 정합성은 지키지만 10만 요청에 88.97초, p99 87초가 걸렸고 기본 힙(512MB)에서 OutOfMemoryError까지 났습니다. "원자적으로, 그리고 빠르게" 중 '빠르게'를 못 지켰습니다.

해결

재고 확인과 차감을 Redis Lua Script 하나로 묶어 락 없이 원자성을 보장하고, 결과는 Redis Stream으로 넘겨 DB에 비동기로 쌓은 뒤 배치로 정합성을 검증합니다.

발급 파이프라인

발급 요청
10만 동시

Lua Script
확인+차감 원자

Redis Stream
발급 결과

Consumer
읽기 전용

Writer
JDBC 배치 insert

MySQL
발급 이력

검증 배치
60초 대조

담당 업무

영역	내용
Lua Script 제안	재고 확인과 차감을 따로 보내면 사이에 다른 요청이 끼어들어 별도 락이 필요하고 왕복도 늘어난다고 판단. 하나의 Lua Script로 묶자고 제안해 채택, 발급 파이프라인의 핵심이 됨
DB 적재 병목	Consumer가 블로킹 read와 DB 저장을 한 흐름에서 처리해 적재가 밀리던 문제를, 읽기와 쓰기를 분리하고 쓰기는 별도 스레드에서 모아 JDBC 배치 insert로 저장하도록 개선
동기화 배치	마감된 쿠폰의 실제 발급 이력 건수와 확정 수량을 맞추는 동기화 배치 구현
검증 배치	60초마다 마감 쿠폰을 재대조. 한 번 통과한 쿠폰은 대상에서 제외해 배치 부하가 쌓이지 않게 함
재고 복구	Redis 재고 키 유실 시 DB의 유효 발급 건수(취소·만료 제외)로 재고를 재계산해 복구

Tech

`Java 21` `Spring Boot 4.1` `Spring Data JPA` `MySQL · Flyway` `Redis Lua` `Redis Stream` `Kafka (비교용)` `k6` `React 19 · TypeScript` `Docker Compose`

FEATURED PROJECT · 02 - DEEP DIVE

동시성 제어 방식 비교와 정합성 검증

같은 조건(가상 스레드 10만 개, 재고 1만 개)에서 방식을 바꿔 가며 측정하고, 수치로 설계를 결정했습니다.

0.18s
10만 요청 처리 (Lua 배치)

58만/s
초당 처리량

0건
초과 발급 (전 방식)

방식별 비교

방식	처리 시간	비고
DB 비관적 락	**88.97초**	p99 87초, 기본 힙 512MB에서 OOM
Lua Script (낱개)	**10.11초**	비관적 락 대비 약 9배 빠름
Kafka 배치	**7.83초**	-
Lua Script (500건 배치) ✓	**0.18초**	3회 측정 0.15~0.21초, Kafka 대비 약 43배, 피크 힙 81MB로 최저

모든 방식에서 최종 재고 `0` · 성공 발급 `10,000`건 · 초과 발급 `0`건 확인

내가 한 일

- **Lua Script로 원자성 확보** - Redis는 Lua Script 하나를 실행하는 동안 다른 명령을 끼워 넣지 않으므로, 락 없이 엔진 수준에서 원자성이 보장되고 왕복은 한 번으로 줄어듭니다
- **읽기와 쓰기 분리** - Consumer는 Stream 읽기에만 집중하고, DB 쓰기는 별도 스레드가 모아서 배치로 처리해 DB 왕복 횟수를 건별 저장 대비 크게 줄였습니다
- **비동기 구조의 정합성** - Redis와 DB가 어긋날 수 있는 구조라 동기화 배치, 60초 검증 배치, 재고 키 유실 시 DB 기반 복구를 직접 만들었습니다
- **일부러 틀려 보기** - 발급 수량을 1건 조작한 테스트에서 검증 배치가 불일치를 바로 잡아내는 것을 확인했습니다

배운 것

Redis만으로는 부족하다
명령을 나눠 보내면 그 사이가 빈틈입니다. 원자성은 단위로 묶어야 생깁니다.

비동기는 검증과 한 세트
빠르게 쌓는 만큼 어긋남을 찾아 고치는 장치가 같이 있어야 합니다.

측정이 설계를 정한다
같은 조건으로 재야 방식끼리 비교가 됩니다.

FEATURED PROJECT · 03

Proxy Gateway

리버스 프록시·로드밸런싱·오토스케일링·헬스체크의 내부 동작을 Nginx 없이 직접 구현

개인 프로젝트 · 1인 · Go 게이트웨이 + Spring Boot 백엔드 · github.com/UHO99/Simple-Reverse-Proxy-Gateway

문제

처음 만든 게이트웨이는 포트 범위를 고정해 스캔했기 때문에 인스턴스 수가 정적이고, 부하가 늘면 사람이 직접 인스턴스를 늘려야 했습니다.

해결

Docker API로 실행 중인 컨테이너를 찾고, 활성 커넥션으로 부하를 재서 인스턴스를 자동으로 늘리고 줄입니다.

동작 흐름

Discovery	헬스체크	라운드 로빈	부하 측정	평가	Scale
Docker API	goroutine 병렬	atomic counter	ActiveConns	임계값·쿨다운	compose --scale

구현 내용

영역	내용
리버스 프록시	Gin + httputil.ReverseProxy로 모든 경로를 백엔드로 전달, X-Forwarded 헤더 설정
로드밸런서	RWMutex와 atomic counter로 라운드 로빈 선택. 헬스체크는 WaitGroup + goroutine으로 병렬 수행하고 Actuator 상태가 UP인 인스턴스만 풀에 등록
Discovery	포트 스캔 대신 compose 서비스 라벨로 실행 중인 컨테이너를 조회. 기존 Backend 객체를 재사용해 갱신 때마다 부하 카운터가 초기화되지 않게 함
오토스케일러	Discovery(10초)와 스케일 평가(15초) 주기를 분리. 평균 활성 커넥션이 임계값을 넘으면 1대씩 증감, 쿨다운 적용, 스케일 직후 재discovery 예약

트러블슈팅

증상	원인과 해결
Scale In이 전혀 안 됨	오토스케일러 초기 인스턴스 수를 실제 조회 없이 최소값으로 고정해, 2대가 떠 있어도 "1대"로 인식. 시작 시 실제 healthy 인스턴스 수로 동기화하도록 수정
compose build 실패	`build` 키에 이미지 태그를 넣은 실수. `build` (컨텍스트)와 `image` (태그)를 분리하고, 스케일 시 `--no-build` 로 재빌드 차단

배운 것 - 상태를 들고 있는 로직은 실제 인프라 상태와 애플리케이션이 기억하는 상태가 어긋나는 순간 조용히 멈춥니다. 시작 시점에는 항상 실측값으로 동기화해야 합니다.

Tech

Go · Gin · Docker Engine SDK · Spring Boot · Actuator · MySQL · Docker Compose

FEATURED PROJECT · 04

ChatFlow

인증과 실시간 통신을 각각 성격에 맞는 언어로 분리한 실시간 채팅 서비스

개인 프로젝트 · 1인 · Spring Boot 인증 서버 + Go 채팅 서버 · github.com/UHO99/ChatFlow

문제

인증은 요청-응답 기반의 전형적인 CRUD·보안 처리이고, 채팅은 다수 클라이언트가 장시간 커넥션을 유지합니다. 성격이 다른 두 일을 한 서버에 두면 어느 한쪽의 강점을 살리기 어렵습니다.

해결

인증은 검증된 Spring Security로, 채팅은 커넥션당 자원이 적은 Go goroutine으로 나누고, 두 서버는 JWT 하나로 연동했습니다.

요청 흐름

회원가입·로그인	JWT 발급	Nginx	JWT 검증	Hub / Room	브로드캐스트
SpringAuth	Spring Security	리버스 프록시	Go 미들웨어	커넥션 관리	WebSocket

구현 내용

영역	내용
인증 서버	Spring Security 필터 체인과 UserDetailsService 구조로 회원가입·로그인·JWT 발급·토큰 블랙리스트 관리
채팅 서버	Gin + WebSocket으로 Hub/Room 구조를 만들고, goroutine과 channel로 별도 스레드풀 관리 없이 다수 커넥션의 브로드캐스트 처리
서버 간 연동	Spring(JJWT)이 발급한 토큰을 Go(golang-jwt) 미들웨어에서 검증. 두 라이브러리 간 JWT 파싱 포맷을 맞춰 호환 문제 해결
데이터	PostgreSQL 스키마와 마이그레이션, Go 쪽은 pgx로 쿼리
인프라	Postgres·SpringAuth·GoChat·Nginx를 Docker Compose로 한 번에 기동, 방 참가 플로우와 테스트용 프론트 작성

배운 것

도구는 일의 성격으로 고른다

요청-응답과 장시간 커넥션은 필요한 강점이 다릅니다.

경계는 토큰 하나로

인증 책임을 한곳에 두고, 다른 서버는 검증만 하게 했습니다.

Tech

`Java 21` `Spring Boot 4.1` `Spring Security` `JJWT` `Go 1.25` `Gin` `WebSocket` `golang-jwt` `pgx` `PostgreSQL` `Nginx` `Docker Compose`

FEATURED PROJECT · 05

Study Room Mate

스터디룸·공간 예약부터 결제까지 처리하는 예약 통합 관리 시스템

LG U+ 유레카 조별 프로젝트 · 3인 팀 · 예약·결제 도메인 (저장소 최다 커밋) · github.com/rmsckd1640/study-room-mate

문제

같은 방·같은 시간에 예약이 동시에 들어오면 중복 예약이 생길 수 있고, 결제를 시작만 하고 끝내지 않은 예약은 자리를 영구히 점유합니다.

해결

예약 시 방을 비관적 쓰기 락으로 잡아 동시 예약을 막고, 결제 상태에 따라 예약 상태를 전환하며, 미완료 예약은 스케줄러가 주기적으로 취소합니다.

예약 · 결제 흐름

방 조회	예약	PENDING	Toss 결제	확정	스케줄러
목록·현황	PESSIMISTIC_WRITE	결제 대기	Server-to-Server 승인	상태 전환	10분마다 정리

담당 업무

영역	내용
예약 동시성	예약 시 Room을 findByIdForUpdate(PESSIMISTIC_WRITE)로 조회해 동시 요청의 중복 예약 차단, 예약 상태 전환 메서드 설계
결제 연동	Toss Payments 위젯 연동과 서버 간 결제 승인, 승인 중 발생 가능한 오류와 successUrl 미전송 처리, 결제에 따른 예약 상태 전환
점유 해제	PENDING·READY 상태로 남아 자리를 영구 점유하는 예약을 스케줄러가 10분마다 CANCELED로 변경
구조 개선	WebFlux 의존을 RestClient로 교체, 서비스 레이어의 try-catch를 걷어내고 GlobalExceptionHandler로 오류 처리 일원화, 예약·결제 레이어 리팩토링
테스트	Repository JUnit 단위 테스트, 관리자 방 생성 → 회원 로그인 → 방 조회 → 예약 시나리오 MVC 테스트, JaCoCo 커버리지
프론트·배포	React + Vite 프론트 초기 구조, GitHub Pages 배포 워크플로

배운 것

락의 대상은 경합하는 자원

예약 레코드가 아니라 방을 잠가야 중복이 막힙니다.

미완료 상태도 설계 대상

결제를 멈춘 사용자의 예약을 누가 언제 풀지까지 정해야 합니다.

Tech

Java 21 · Spring Boot 4.1 · Spring Data JPA · Spring Security · JWT · MySQL · Toss Payments · RestClient · JUnit · JaCoCo · React · Vite · GitHub Pages

OTHER PROJECT

よちよち (YochiYochi)

일본 영유아가 재미있게 글을 학습할 수 있도록 만든 학습 사이트

영진전문대 졸업 프로젝트 · 프론트엔드 담당 · github.com/YunYeongUn/YochiYochiServer

"よちよち"는 한국어로 "아장아장"이라는 뜻입니다. 백엔드는 Laravel, 프론트는 Vue.js로 제작했고 그중 프론트를 담당했습니다.

Vue.js Laravel

CERTIFICATION

JLPT N1 153점 · 2023.12

CONTACT

Email	uh787355@gmail.com
Phone	010-9984-3865
GitHub	github.com/UHO99
Blog	umjuho9984.tistory.com